

# WGU C949 Study Guide

Oct 3, 2026 · @John

## Exam blueprint

You pass C949 by passing one proctored, multiple-choice exam: the Objective Assessment (OA). Applying algorithms is the biggest share of the exam at 40%. Explaining algorithms (29%) feels hardest, because it means tracing pseudocode by hand.

| Competency                       | Weight | What it covers                                                                                              | zyBooks chapters                       |
|----------------------------------|--------|-------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Applies Algorithms               | 40%    | Variables, operators, precedence, loops, branching, classes, list methods, searching, sorting, Big-O        | 2.1, 3.1, 3.8, 11, 13-15, 17, 18.3, 20 |
| Determines Data Structure Impact | 31%    | Arrays, linked lists, stacks, queues, dequeues, hash tables, trees, heaps, sets, graphs, ADTs, dictionaries | 1.1-1.6, 2-5, 7-9, 14.5                |
| Explains Algorithms              | 29%    | Recursion, linear and binary search, sorting, Big-O best/average/worst case, classes                        | 1, 11, 20, 21                          |

Also review the website links in zyBooks chapter 1.

The questions test three different skills, and each one takes a different kind of practice:

- **Knowing the vocabulary.** A typical question: "Which data structure is usually used to build a priority queue?" (Answer: a heap.) Learn these from the tables in this guide.
- **Knowing the speeds.** A typical question: "What is quicksort's worst-case Big-O?" (Answer:  $O(n^2)$ .) Practice by rewriting the Big-O tables from memory until you get them right.
- **Tracing code by hand.** A typical question: "What does this pseudocode output?" Practice by tracing on your whiteboard, writing down every variable's value at every step.

**Study plan:** Take the pre-assessment before you study, and use its coaching report to find your weak areas. Learn the programming basics and data structures first, then practice tracing every day. Retake the pre-assessment once you can trace code without guessing.

For hands-on lessons, Python exercises, and pseudocode tracing drills, see the [Python Practice](#) tab.

## Programming basics

These questions are easy points on the exam, and the pre-assessment asks about them often. The idea behind all of them: a program stores values in named boxes called **variables**, and each box has a **type** that says what kind of value it can hold.

### Data types

Python figures out a variable's type from the value you assign, and `type(x)` tells you what it is. Match the type to the kind of value: whole numbers are **int**, numbers with a decimal point are **float**, True/False is **bool**, and text is **str**.

| Python type           | Holds                                              | Example                                  |
|-----------------------|----------------------------------------------------|------------------------------------------|
| <code>bool</code>     | True or False                                      | <code>done = True</code>                 |
| <code>int</code>      | Whole numbers, any size                            | <code>count = 10</code>                  |
| <code>float</code>    | Numbers with a decimal point                       | <code>price = 3.14</code>                |
| <code>str</code>      | Text; a single character is just a 1-character str | <code>name = "Hello", grade = 'A'</code> |
| <code>list</code>     | Ordered collection you can change                  | <code>nums = [1, 2, 3]</code>            |
| <code>tuple</code>    | Ordered collection you can't change                | <code>point = (3, 4)</code>              |
| <code>dict</code>     | Key → value pairs                                  | <code>ages = {"Ann": 30}</code>          |
| <code>set</code>      | Unique items, no order                             | <code>tags = {"a", "b"}</code>           |
| <code>NoneType</code> | No value at all                                    | <code>result = None</code>               |

Mixing types: an int combined with a float gives a float ( `2 * 1.5` is `3.0` ), and `/` always gives a float, even for whole numbers ( `6 / 2` is `3.0` ).

**Example question 1.** What data type does `mystery(4)` return?

```
def mystery(r):
    num_units = r
    return num_units * 3.14
```

**Answer: float.** `num_units` is 4, an int, but the function multiplies it by 3.14. An int times a float always gives a float, so it returns 12.56, a float.

**Example question 2.** What data type is every value in `["AF", "71", "BC", "157"]` ?

**Answer: str.** Every value is in quotes, so every value is text, even "71" and "157". Numbers in quotes are text, not numbers.

## Key terms

| Term                      | In plain words                                                                                                                                  | Python example                                                                                          |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Assignment vs. comparison | <code>=</code> puts a value in a variable;<br><code>==</code> asks "are these equal?"                                                           | <code>x = 5</code> vs. <code>if x == 5:</code>                                                          |
| Dynamic typing            | A variable's type comes from the value it holds right now, and it can change                                                                    | <code>x = 5</code> , then <code>x = "five"</code>                                                       |
| Reference                 | A variable holds <i>where</i> an object lives in memory, like a street address instead of the house. Two variables can point to the same object | <code>a = [1, 2]</code> , <code>b = a</code> , then <code>b.append(3)</code> changes <code>a</code> too |
| None                      | "No value"; points to nothing                                                                                                                   | <code>result = None</code>                                                                              |
| Memory allocation         | Reserving memory for a new object; Python does it for you whenever you create one                                                               | <code>node = Node(5)</code>                                                                             |
| Sequential allocation     | Items stored side by side in one block, like seats in a row                                                                                     | A Python <code>list</code>                                                                              |
| Linked allocation         | Items scattered in memory, each holding a reference to the next, like a scavenger hunt                                                          | A linked list of <code>Node</code> objects                                                              |

| Term               | In plain words                                                                 | Python example                                                               |
|--------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Garbage collection | Python automatically frees the memory of objects that nothing uses anymore     | Happens on its own                                                           |
| Reference count    | How many variables point at an object; when it hits 0, Python frees the object | <code>sys.getrefcount(x)</code>                                              |
| Constructor        | The method that sets up a new object                                           | <code>def __init__(self, data):</code>                                       |
| Nested if          | An if statement inside another if statement                                    | <code>if x &gt; 0:</code> with <code>if x &lt; 10:</code> indented inside it |

## Order of operations

Parentheses first, then exponents ( `**` ), then `*` / `//` `%`, then `+` `-`, then comparisons ( `<` `==` `...` ), then `not`, `and`, `or`. So `2 + 3 * 4` is 14, not 20.

Know Python's three division operators: `7 / 2` is 3.5, `7 // 2` is 3 (drops the decimal), and `7 % 2` is 1 (the remainder).

## Big-O, explained simply

Big-O answers one question: **if the data gets bigger, how much more work does the algorithm do?** It doesn't measure seconds or care how fast your computer is; it counts steps. In Big-O, **n** stands for the number of items the algorithm works on.

| Big-O | Name     | Everyday picture                                 | Steps when n = 1,000 | Typical example                          |
|-------|----------|--------------------------------------------------|----------------------|------------------------------------------|
| O(1)  | Constant | Opening a book to a page number you already know | 1                    | Array index, stack push/pop, hash lookup |

| Big-O         | Name         | Everyday picture                                                                 | Steps when n = 1,000 | Typical example                                 |
|---------------|--------------|----------------------------------------------------------------------------------|----------------------|-------------------------------------------------|
| $O(\log n)$   | Logarithmic  | The number-guessing game: "higher or lower?" cuts the options in half each guess | about 10             | Binary search                                   |
| $O(n)$        | Linear       | Checking every person in a line for your friend                                  | 1,000                | Linear search, one loop over a list             |
| $O(n \log n)$ | Linearithmic | Sorting a deck by splitting it into small piles and merging them back            | about 10,000         | Merge sort, heap sort, average quicksort        |
| $O(n^2)$      | Quadratic    | Everyone in a room shaking hands with everyone else                              | 1,000,000            | Nested loops; bubble, selection, insertion sort |
| $O(2^n)$      | Exponential  | Each extra item doubles the work                                                 | astronomically large | Naive recursive Fibonacci                       |

The gaps between rows are what matter. At 1,000 items, an  $O(n^2)$  algorithm does 100,000 times more work than an  $O(\log n)$  one.

If Big-O is giving you trouble, the **Big-O Made Easy** tab walks through a five-step decoder, 15 worked examples, the class worksheet, and practice problems.

### Reading Big-O off code

1. **One loop over the data**  $\rightarrow O(n)$ .
2. **A loop inside a loop over the same data**  $\rightarrow$  multiply:  $O(n) \times O(n) = O(n^2)$ .
3. **Two loops one after the other**  $\rightarrow$  add, then keep the bigger:  $O(n) + O(n) = O(n)$ .
4. **A loop whose counter doubles or halves** ( `i = i * 2` )  $\rightarrow O(\log n)$ .
5. **Two different inputs** (loop over list A, inside a loop over list B)  $\rightarrow O(n \times m)$ .
6. **Drop constants and small terms**:  $3n^2 + 5n + 2$  is just  $O(n^2)$ , because when n is huge the  $n^2$  part swamps everything else.

## Best, average, and worst case

The same algorithm can be lucky or unlucky. Linear search finds the item on the first try (best case,  $O(1)$ ) or checks everything (worst case,  $O(n)$ ). Exam questions mean **worst case** unless they say otherwise.

**Space complexity** is the same idea for memory: how much *extra* memory the algorithm needs. Merge sort needs an extra  $O(n)$  copy; bubble sort works in place, using only  $O(1)$  extra memory.

## Tracing pseudocode and recursion

The exam shows code in **pseudocode**, a plain-English style that is not exactly Python. Tracing means playing computer: you run the code by hand and write down every variable's value after every step. Skip a single step and you'll likely pick a wrong answer, so always trace on paper, never in your head.

### How to trace

1. Draw a table with one column per variable, plus an "Output" column.
2. Go line by line. Each time a variable changes, write its new value on a new row.
3. At every loop or if, write the condition and whether it was true or false.
4. Stop when the code ends, then read the Output column.

### Worked example: nested loops

```
n = 2
i = 2
while i <= 2
    j = 0
    while j <= n
        Put j to output
        j = j + 1
    i = i + 1
```

| Step | Check            | Action                        |
|------|------------------|-------------------------------|
| 1    | $i = 2 \leq 2$ ✓ | Enter outer loop, set $j = 0$ |
| 2    | $j = 0 \leq 2$ ✓ | Output 0, $j = 1$             |

| Step | Check            | Action                   |
|------|------------------|--------------------------|
| 3    | $j = 1 \leq 2$ ✓ | Output 1, $j = 2$        |
| 4    | $j = 2 \leq 2$ ✓ | Output 2, $j = 3$        |
| 5    | $j = 3 \leq 2$ ✗ | Exit inner loop, $i = 3$ |
| 6    | $i = 3 \leq 2$ ✗ | Exit outer loop          |

Output: 0 1 2. The classic mistake is stopping after 1 and forgetting that `<=` ("less than or equal to") includes the end value, 2.

## Recursion, explained simply

A recursive function solves a problem by calling itself on a **smaller version of the same problem**. Think of Russian nesting dolls: open one, find a smaller one inside, and keep going until you reach the tiny solid doll that doesn't open. That smallest doll is the **base case**; it answers directly and stops the chain. Without a base case, the calls never stop and the program crashes with a "stack overflow" error.

```
Factorial(n)
  if n == 1
    return 1                // base case
  return n * Factorial(n - 1) // recursive case: smaller problem
```

Trace `Factorial(3)` by going down, then coming back up:

1. `Factorial(3)` waits for  $3 \times \text{Factorial}(2)$
2. `Factorial(2)` waits for  $2 \times \text{Factorial}(1)$
3. `Factorial(1)` hits the base case and returns 1
4. `Factorial(2)` now returns  $2 \times 1 = 2$
5. `Factorial(3)` now returns  $3 \times 2 = 6$

Each waiting call sits on the **call stack**, which is why recursion uses memory proportional to how deep it goes. Merge sort, quicksort, binary search, and tree traversals are all naturally recursive.

## Searching and sorting

For each algorithm, know its best, average, and worst case. Also be able to recognize it from its code, or from an array partway through being sorted.

### Searching

**Linear search** is checking every drawer until you find your keys: it works on any data, but it's slow. **Binary search** is looking up a word in a dictionary: open to the middle, decide which half the word is in, and repeat. It is very fast but only works if the data is **sorted** and you can jump to any position (an array, not a linked list).

| Algorithm     | How it works                                                    | Needs sorted data? | Best   | Worst       |
|---------------|-----------------------------------------------------------------|--------------------|--------|-------------|
| Linear search | Check each item in order until found                            | No                 | $O(1)$ | $O(n)$      |
| Binary search | Check the middle, keep only the half that could hold the target | Yes                | $O(1)$ | $O(\log n)$ |

Useful number: binary search needs at most about  $\log_2 n + 1$  checks. 1,024 items take at most 11; a million take about 20.

### Sorting, one picture each

- **Bubble sort:** walk the list swapping neighbors that are out of order. The biggest value "bubbles" to the end each pass. Stops early if a pass makes no swaps.
- **Selection sort:** find the smallest card, put it first; find the next smallest, put it second. Always  $O(n^2)$ , even on sorted data.
- **Insertion sort:** how you sort a hand of cards. Take the next card and slide it left into its spot among the cards you already sorted. Very fast on nearly sorted data.
- **Merge sort:** split the list in half again and again until each piece has one item, then merge pieces back together in order. Always  $O(n \log n)$ , but needs extra memory.
- **Quicksort:** pick a **pivot**, put smaller items on its left and larger on its right (**partition**), then repeat on each side. Usually the fastest; a bad pivot (like the last item of already-sorted data) makes it  $O(n^2)$ .
- **Heap sort:** build a heap, then repeatedly move the root (the largest value) to the end of the list.

- **Radix sort:** no comparisons. Group the numbers by their ones digit, then by their tens digit, then by their hundreds digit.
- **Bucket sort:** spread values into range buckets (0–9, 10–19, ...), sort each bucket, then join them.

| Sort      | Best          | Average           | Worst         | Extra memory | How to spot it in code                                   |
|-----------|---------------|-------------------|---------------|--------------|----------------------------------------------------------|
| Bubble    | $O(n)$        | $O(n^2)$          | $O(n^2)$      | $O(1)$       | Swaps adjacent pairs; largest moves to the end each pass |
| Selection | $O(n^2)$      | $O(n^2)$          | $O(n^2)$      | $O(1)$       | Finds the minimum, swaps it to the front                 |
| Insertion | $O(n)$        | $O(n^2)$          | $O(n^2)$      | $O(1)$       | Shifts items right to insert into a sorted part          |
| Shell     | $O(n \log n)$ | $\sim O(n^{1.5})$ | $O(n^2)$      | $O(1)$       | Insertion sort on items a "gap" apart; gap shrinks to 1  |
| Merge     | $O(n \log n)$ | $O(n \log n)$     | $O(n \log n)$ | $O(n)$       | Split in half, recurse, merge                            |
| Quick     | $O(n \log n)$ | $O(n \log n)$     | $O(n^2)$      | $O(\log n)$  | Look for the words "pivot", "partition", or "split"      |
| Heap      | $O(n \log n)$ | $O(n \log n)$     | $O(n \log n)$ | $O(1)$       | Builds a heap, removes the root repeatedly               |
| Radix     | $O(nk)$       | $O(nk)$           | $O(nk)$       | $O(n + k)$   | Digit by digit; $k$ = digits in the largest number       |
| Bucket    | $O(n + k)$    | $O(n + k)$        | $O(n^2)$      | $O(n + k)$   | Values spread into buckets sorted separately             |

**Spotting a sort from a half-sorted array:** if the front holds the *smallest* items in final order, it's selection sort. If the front is sorted but those items may still change, it's insertion sort. If the *largest* items are piling up at the end, it's bubble sort.

**Stable** means equal values keep their original order. Bubble, insertion, merge, radix, and bucket sort are stable; selection, quick, and heap sort are not.

**For exam answers,** the class Big-O worksheet says to use  $O(n)$  for radix sort and  $O(n)$  as bucket sort's average. The exam-ready best/average/worst table is in [Big-O Made Easy](#).

## Lists, stacks, queues, and ADTs

The pre-assessment asks about these often, especially what `push()` and `pop()` do.

## ADT vs. data structure

An **abstract data type (ADT)** is a promise about *what* you can do, like a restaurant menu. A **data structure** is *how* it's actually done, like the kitchen. "Queue" is an ADT (first in, first out); a linked list is one data structure that can deliver that promise. Many questions are just "which structure usually builds this ADT?"

## Stacks and queues, in pictures

- **Stack = a stack of plates.** You add to the top and take from the top. Last in, first out (**LIFO**). Used for undo, the browser back button, matching parentheses, and the call stack in recursion.
- **Queue = a line at the store.** Join at the back, leave from the front. First in, first out (**FIFO**). Used for print jobs, task scheduling, and breadth-first search.
- **Deque ("deck") = a line where people can join or leave at either end.**
- **Priority queue = an emergency room.** The most urgent patient goes first, no matter who arrived first. Built with a heap.

| Structure      | Rule        | pop() / dequeue removes                                                        |
|----------------|-------------|--------------------------------------------------------------------------------|
| Stack          | LIFO        | The newest item (the top)                                                      |
| Queue          | FIFO        | The oldest item (the front)                                                    |
| Priority queue | By priority | The highest-priority item; in Python's <code>heapq</code> , the smallest value |

Quick check: push 1, 2, 3 onto a stack, then pop. You get 3. Enqueue 1, 2, 3 into a queue, then dequeue. You get 1.

## Common ADTs

| ADT   | Rules in plain words                    | Usually built on | Python syntax                                                              |
|-------|-----------------------------------------|------------------|----------------------------------------------------------------------------|
| List  | Ordered, numbered positions, changeable | Dynamic array    | <code>[]</code> , <code>append(x)</code> , <code>pop()</code> removes last |
| Tuple | Ordered, numbered, can't be changed     | Array            | <code>()</code> , indexing and slicing                                     |

| ADT              | Rules in plain words                                              | Usually built on                                                | Python syntax                                                                                                                      |
|------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Stack            | Add and remove at the top only                                    | Array or linked list                                            | <code>append(x)</code> = push, <code>pop()</code> = pop, <code>s[-1]</code> = peek                                                 |
| Queue            | Add at the back, remove from the front                            | Linked list                                                     | <code>append(x)</code> = enqueue, <code>pop(0)</code> = dequeue; <code>collections.deque</code> + <code>popleft()</code> is faster |
| Deque            | Add or remove at both ends                                        | Linked list                                                     | <code>append</code> , <code>appendleft</code> , <code>pop</code> , <code>popleft</code>                                            |
| Bag              | No order, duplicates allowed                                      | Array or linked list (zyBooks); the class guide says hash table | <code>add(x)</code> , <code>remove(x)</code>                                                                                       |
| Set              | No order, no duplicates                                           | Hash table or BST                                               | <code>set()</code> , <code>add</code> , <code>remove</code> , <code>a &amp; b</code> = items in both                               |
| Priority queue   | Highest priority leaves first                                     | Heap                                                            | <code>heapq.heappush</code> , <code>heapq.heappop</code>                                                                           |
| Dictionary (map) | Each unique key points to a value, like a word and its definition | Hash table or BST                                               | <code>{}</code> , <code>d[k]</code> , <code>keys()</code> , <code>values()</code> , <code>items()</code> , <code>del d[k]</code>   |

The class guide and zyBooks disagree about what usually builds a bag. Check the ADT table in zyBooks chapter 1 and use its answer on the exam.

## Arrays vs. linked lists

An **array** is like numbered lockers in a row: you can open locker 57 instantly, but squeezing a new locker into the middle means shifting every locker after it. A **linked list** is like a scavenger hunt: each clue (node) tells you where the next one is. Adding a clue anywhere is just rewriting one pointer, but to reach clue 57 you must follow the first 56.

| Structure          | Memory                    | Get the item at position i | Add/remove at end                                                             | Add/remove at front or middle                     |
|--------------------|---------------------------|----------------------------|-------------------------------------------------------------------------------|---------------------------------------------------|
| Array              | One contiguous block      | $O(1)$                     | $O(1)$ (amortized)                                                            | $O(n)$ : items shift                              |
| Singly linked list | Nodes point to the next   | $O(n)$ : walk from head    | $O(1)$ with a tail pointer to add; $O(n)$ to remove (need the node before it) | $O(1)$ once you're at the spot; $O(n)$ to find it |
| Doubly linked list | Nodes point next and back | $O(n)$                     | $O(1)$                                                                        | $O(1)$ once you're at the spot; $O(n)$ to find it |

A few extra facts: an array that fills up gets copied into a bigger one, which is why appending is  $O(1)$  *on average* (amortized). A **sentinel** (dummy) head node removes the special case of an empty list. Stacks and queues add and remove in  $O(1)$  and are not accessed by index.

## Hash tables

A hash table is a **coat check**. You hand over your coat (the value) and get a ticket number (the hash). To get your coat back you don't search every hook; you go straight to the hook on your ticket. That's why insert, search, and delete are  $O(1)$  on average.

- **Hashing**: turning a key into a slot number (an array index).
- **Hash function**: the rule that does it. The exam's favorite is **modulo hashing**:  $\text{slot} = \text{key} \% \text{table size}$ . Key 47 in a 10-slot table goes to slot 7.
- **Collision**: two keys land on the same slot (17 and 47 both go to slot 7). Collisions are normal; the table needs a plan for them.
- **Chaining**: each slot holds a small list, and colliding items share it, like two coats on one hook. This is the method the class guide covers.
- **Worst case  $O(n)$** : if every key collides, you're back to searching one long list.

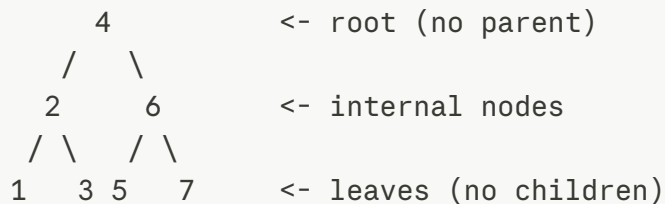
Worked example with chaining, table size 10: insert 12, 22, 35.  $12 \% 10 = 2$  and  $22 \% 10 = 2$ , so slot 2 holds [12, 22]; 35 goes to slot 5.

## Beyond the class guide (lower priority)

zyBooks also describes **open addressing**, where a colliding item looks for another empty slot instead of sharing. **Linear probing** tries the next slot ( $i+1, i+2, \dots$ ); **quadratic probing** jumps by 1, 4, 9, ...; **double hashing** uses a second hash for the jump size. The **load factor** ( $\text{items} \div \text{slots}$ ) measures fullness, and a table that gets too full is resized and every item rehashed. Know the terms in case they appear, but they're not core.

## Trees and heaps

A tree is a **family tree turned upside down**: one ancestor at the top, children below. In a **binary tree**, each node has at most two children.



- **Root**: the top node. **Leaf**: a node with no children. **Edge**: the line between parent and child.
- **Depth** of a node: edges from the root down to it (root = 0). **Height** of the tree: edges on the longest root-to-leaf path. The tree above has height 2; a single node has height 0.
- **Full**: every node has 0 or 2 children. **Complete**: every level is filled except maybe the last, which fills left to right. **Perfect**: completely filled, all leaves on the same level (the tree above).

## Binary search trees

A **BST** adds one rule: everything in a node's left subtree is smaller, everything in its right subtree is larger. Searching is the number-guessing game again: compare, go left or right, repeat. The tree above is a BST.

- Search, insert, and delete take  $O(\text{height})$ . In a balanced tree, that's  $O(\log n)$ . But if you insert already-sorted data (1, 2, 3, 4, ...), each new node hangs off the right of the one before it, and the "tree" turns into a linked list:  $O(n)$ . Self-balancing trees (AVL, red-black) prevent this.
- Deleting a node: a leaf just goes away; a node with one child is replaced by that child; a node with two children is replaced by its **in-order successor** (the smallest value in its right subtree).

## Traversals: the order you visit nodes

The name tells you when you visit the node itself: **pre-order** = before its children, **in-order** = in between, **post-order** = after.

| Traversal   | Order                               | Result on the tree above                      |
|-------------|-------------------------------------|-----------------------------------------------|
| Inorder     | Left, node, right                   | 1, 2, 3, 4, 5, 6, 7 (always sorted for a BST) |
| Preorder    | Node, left, right                   | 4, 2, 1, 3, 6, 5, 7                           |
| Postorder   | Left, right, node                   | 1, 3, 2, 5, 7, 6, 4                           |
| Level-order | Row by row, top down (uses a queue) | 4, 2, 6, 1, 3, 5, 7                           |

## Heaps

A heap is a **tournament bracket**: the winner always sits at the top. In a **max-heap** every parent is  $\geq$  its children, so the largest value is the root; in a **min-heap** every parent is  $\leq$  its children. Unlike a BST, there's no left/right ordering between siblings. Heaps are how priority queues are built.

- **Insert**: put the new value in the next open leaf spot, then swap it up with its parent while it beats the parent.  $O(\log n)$ .
- **Remove the root**: move the last leaf into the root's place, then swap it down with its larger child (max-heap) until it settles.  $O(\log n)$ .
- **Peek** at the root:  $O(1)$ .

Because a heap is a complete tree, it can be stored in a plain list with no pointers: each node's position in the list tells you where its children and parent are. Learn **both** numbering conventions. The class materials store the heap in a list called `heapList` with the root at index 1, while Python lists usually put the root at index 0.

| Root at                  | Left child of i | Right child of i | Parent of i    |
|--------------------------|-----------------|------------------|----------------|
| Index 1 (class heapList) | $2i$            | $2i + 1$         | $i // 2$       |
| Index 0 (Python lists)   | $2i + 1$        | $2i + 2$         | $(i - 1) // 2$ |

Example max-heap `[9, 8, 5, 1, 3, 2]` with the root at index 0: 9's children are at indexes 1 and 2 (8 and 5); 8's children are at 3 and 4 (1 and 3).

**Tries** (beyond the class guide) store words one letter per node, like an autocomplete dictionary.

## Graphs

A graph is a **map of cities and roads**: cities are **vertices** (nodes) and roads are **edges**, written as pairs  $(u, v)$ . Unlike a tree, a graph has no root, any vertex can connect to any other, and you can drive in circles (**cycles**).

- **Directed graph**: one-way streets; an edge goes from  $u$  to  $v$  only.
- **Undirected graph**: two-way streets.
- **Weighted graph**: each road has a cost, like miles or minutes.
- **Adjacent vertices**: two cities joined directly by one road.
- **Path**: a route through a sequence of edges.

Two ways to store a graph: an **adjacency list** gives each vertex a list of its neighbors (compact, best when roads are few); an **adjacency matrix** is a  $V \times V$  grid of yes/no (instant edge check, but uses  $V^2$  space).

### Beyond the class guide (lower priority)

The class guide stops at graph vocabulary; the algorithms below mostly belong to the next course, C950. Know the one-liners in case a question names them:

- **Breadth-first search (BFS)** explores in rings outward, using a queue; it finds the path with the fewest edges.
- **Depth-first search (DFS)** goes as deep as possible before backing up, using a stack or recursion.
- **Dijkstra's algorithm** finds the shortest route from one start vertex to all others when no weight is negative.
- **Topological sort** orders tasks so every prerequisite comes first.
- **Minimum spanning tree** (Kruskal, Prim) connects every vertex with the least total edge weight.

## Structures at a glance

"Which structure fits?" questions come down to three things: its shape, whether you can jump to an item, and what keeps it in order.

| Structure          | Shape                          | Can you jump to an item?       | Ordered by                  | Search                             |
|--------------------|--------------------------------|--------------------------------|-----------------------------|------------------------------------|
| Array              | Linear                         | Yes, by index                  | Position                    | $O(n)$ ; $O(\log n)$ if sorted     |
| Linked list        | Linear                         | No, walk from the head         | Position                    | $O(n)$                             |
| Stack / queue      | Linear                         | No, only the top or front      | Arrival time                | Not meant for searching            |
| Hash table         | Key $\rightarrow$ slot         | Yes, by key                    | Not ordered                 | $O(1)$ average                     |
| Binary search tree | Hierarchical                   | No, walk down from the root    | Value (left < node < right) | $O(\log n)$ balanced, $O(n)$ worst |
| Heap               | Hierarchical, stored in a list | Only the root (the max or min) | Parent vs. child            | $O(1)$ for the max/min only        |
| Graph              | Network                        | No, traverse                   | Not ordered                 | $O(V + E)$                         |

Arrays, linked lists, stacks, and queues are **linear** (items in a line). Trees and heaps are **hierarchical** (parent/child levels). Graphs are **networks**.

## Common exam traps

- **Stack vs. queue:** undo, back button, and recursion use a stack; print jobs and waiting lines use a queue.
- **Binary search on unsorted data** doesn't just run slowly; it gives wrong answers.
- **A BST's worst case is  $O(n)$** , not  $O(\log n)$ . It happens when sorted data is inserted.
- **Hash tables are  $O(1)$  on average**; the worst case is  $O(n)$ .
- **Quicksort's worst case is  $O(n^2)$** . Only merge sort and heap sort guarantee  $O(n \log n)$ .
- **Selection sort is  $O(n^2)$  even on sorted input**; bubble and insertion sort drop to  $O(n)$ .
- **Height counts edges:** a single-node tree has height 0.

- **A heap is not a BST:** siblings have no order between them.
- **Loop bounds:** `while j <= n` runs one more time than `while j < n`. Trace it.
- `=` vs. `==`: assignment vs. comparison.
- `/` vs. `//`:  $7 / 2 = 3.5$ , but  $7 // 2 = 3$ .

## Sources

- C949 course study guide from the class resources (competency weights, zyBooks chapter map, worked trace, programming basics)
- [Fennie: WGU C949 study guide](#) (course scope)
- [Online Degree Blogs: How to pass C949](#) (topic list)
- Complexity values are standard textbook figures; check them against your zyBooks chapter tables.